PROJECT-BASED LEARNING

PHASE-II EVALUATION

ScamShield: Explainable Multimodal Scam and Phishing Detection

Team BLAZE — DSCPP-III-2026-T122

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T122
Semester: 3rd
Section: To be confirmed
Domain: AI/ML cybersecurity

Team Members

1. Shivansh Bajaj – 2510370470
2. Ashrey Goel – 2510370302
3. Kangana Agarwal – 2510380099

Mentor

Mr. Pawan Kumar Mishra

Interactions completed: 4

Phase-I Recap & Feedback Incorporated

Phase-I Status

Phase-I established the scam/phishing detection direction. Phase-II integrates SMS/URL baseline models, explainable evidence, C/C++ modules and automated tests.

Review Priorities

- Clarify the full data flow
- Map course concepts to modules
- Separate built and future features

Changes Incorporated

- Documented Python and native roles
- Centralized score and evidence handling
- Added tests and scope limits

Problem Statement & Finalized Objectives

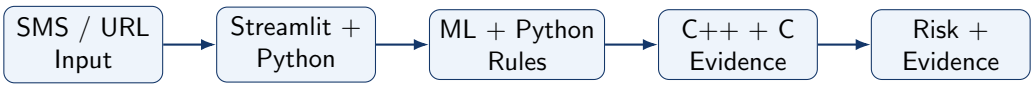
Problem Statement

Scam messages and phishing URLs can mislead users. ScamShield analyses supplied SMS text and URL strings using saved models and explainable rule evidence; it does not open URLs.

Finalized Objectives

1. Analyse suspicious SMS/text and URL-string inputs
2. Combine TF-IDF/Logistic Regression with rules
3. Show score, category, evidence and safety advice
4. Use C++ analysis flags and C evidence storage

Updated System Architecture / Workflow



Major Components

- SMS and URL ML models
- Python rules and C++ flags
- C evidence list and risk engine

Changes Since Phase-I

- Python orchestration path
- C++ flags + C linked list integrated
- Unique-evidence scoring policy

Module Design & Functional Requirements

Major Modules

1. Validate and route inputs
2. Run SMS/URL models
3. Collect Python/C++/C evidence
4. Return score, label and advice

Functional Requirements

- Accept SMS text and/or URL
- Collect model and rule signals
- De-duplicate and prioritize evidence
- Show score, findings and next-step advice

Module Dependency

integration.py combines model probabilities, Python rules and C/C++ evidence. Final score = round(min(100, 0.85 × (mean model probability × 100) + rule points)); unique rule points are capped at 24. Bands: 0–34 Low, 35–69 Suspicious, 70–100 Scam. Heuristic, not calibrated.

Technology Stack & Development Methodology

Technology Stack

- Programming: Python, C and C++
- Database: None in current local prototype
- Framework: Streamlit
- Development Tool: scikit-learn, joblib, ctypes
- Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Module Development
4. Integration
5. Testing & Validation

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	Pointers, dynamic allocation, linked-list traversal, evidence-ID de-duplication and priority ordering.	Live C evidence store
OOPs with C++	C++ MessageAnalyzer / URLAnalyzer classes export rule flags to Python through a C-compatible interface.	Native C++ analyzer
Operating Systems	Not claimed as implemented in this project phase.	N/A
DBMS	Not claimed as implemented in this project phase.	N/A

Important: Mention only those concepts that have been **actually implemented** in the project.

Development & Implementation Progress

Module	Status	Work Completed
Streamlit + Python service	Completed	app.py and integration.py orchestrate input and analysis.
SMS / URL ML pipelines	Completed	Saved TF-IDF + Logistic Regression pipelines; local holdout metrics recorded in models/metrics.json.
C++ analyzer + C DSA store	Tested	Native analyzers and linked-list evidence storage are exercised by integration tests; duplicate IDs are merged.
Browser / external validation	Pending	Real Streamlit browser smoke test and independent external/cross-dataset evaluation remain pending.

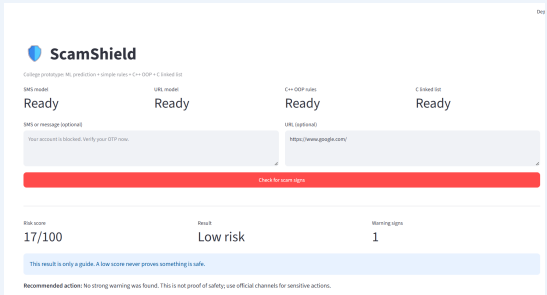
Overall Project Completion: Core service integrated; UI and external validation pending

Working Prototype / Demonstration

Implemented Features

- SMS/text and URL-string analysis
- Saved TF-IDF + Logistic Regression models
- Explainable Python / C++ rule evidence
- C linked-list store and one risk-score engine

Prototype / Output



Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
SMS local hold-out	Binary malicious/benign prediction	98.49% accuracy; F1 95.95%	Pass
URL local hold-out	Phishing/legitimate prediction	99.35% accuracy; F1 99.23%	Pass
Automated integration suite	Models + native analysis + evidence store + risk engine	14 tests passed; end-to-end service case passed	Pass

Results / Validation

SMS (5,971 rows; 1,195 test): Acc 98.49%, P 97.71%, R 94.25%, F1 95.95%. URL (20,000-row sample; 4,000 test): Acc 99.35%, P 100%, R 98.48%, F1 99.23%. Stratified 80/20 split, seed 42. Local holdout only; external generalization remains unverified.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: ScamShield-PBL repository
- README: included in package
- Contributors: verify all three
- Status: source, models and tests included
- Branch: verify on remote

Development Activity

- Python, C and C++ sources included
- 14-test report included
- Each member commits from own account
- Confirm remote matches this ZIP
- README and integration guide included

Requirement: Each member commits from their own GitHub account.

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Shivansh Bajaj 2510370470	Team lead / ML	Architecture, ML models and risk scoring.	Verify
Ashrey Goel 2510370302	C++ / native	C++ analyzers and native bridge.	Verify
Kangana Agarwal 2510380099	Research / QA	Dataset research, tests and documentation.	Verify

Individual Evaluation

Every member must explain their own contribution.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- C/C++ calls from Python
- Duplicate evidence / score control
- Dependencies and browser testing

Solutions / Learning

- C-compatible APIs and fallbacks
- One scorer and 14 automated tests

Phase-III Roadmap

1. Run a real browser smoke test
2. Rebuild native libraries for target OS
3. Validate external data / calibrate scores
4. Plan HTML, screenshot OCR and QR support
5. Finalize report, demo and member contributions

Expected Outcomes & References

Expected Final Outcomes

- Integrated SMS/URL risk triage
- Local metrics; 14 tests pass
- Explainable warnings and user advice
- External validation still needed
- Future screenshot/QR/webpage analysis

References

1. CrossPhire, Almakhamreh & Bozkir (2026), Appl. Sci. 16, 751; DOI 10.3390/app16020751
2. PhiUSIIL, Prasad & Chandra (2024); DOI 10.1016/j.cose.2023.103545
3. SMS Phishing Dataset, Mishra & Soni (2022); DOI 10.17632/f45bkkt8pr.1
4. scikit-learn and Streamlit docs
5. ScamShield-PBL repo and test report

Thank You
Questions & Suggestions